
Semana 27 - Aula 1

Tópico Principal da Aula: Desenvolvimento de Aplicações Robustas com Spring Boot

Subtítulo/Tema Específico: Spring Boot, REST APIs e Persistência de Dados com JPA

Código da aula: [SIS]ANO2C2B4S27A1

Objetivos da Aula:

- Utilizar **Spring Boot** para criar aplicações robustas.
- Configurar e iniciar projetos Spring de forma simplificada com o **Spring Initializr**.
- Criar **Endpoints RESTful** utilizando as anotações do Spring MVC.
- Implementar a camada de persistência de dados usando **Spring Data JPA**.

Recursos Adicionais (Sugestão, pode ser adaptado):

- Caderno para anotações;
- Acesso ao laboratório de informática e/ou internet.

Exposição do Conteúdo:

Referência do Slide: Configuração Inicial e Spring Initializr

- **Definição:** O **Spring Boot** é um projeto do ecossistema Spring que visa simplificar o processo de criação de aplicações Spring. Ele permite a criação de aplicações **autônomas** (que podem ser executadas como JARs) e prontas para produção [ANO2C2B4S27A1.pdf]. O **Spring Initializr** (start.spring.io) é uma ferramenta web que gera a estrutura básica do projeto, incluindo as dependências e configurações iniciais, conforme as escolhas do desenvolvedor.
- **Aprofundamento/Complemento (se necessário):** O Spring Boot se baseia no princípio de "Convenção sobre Configuração", reduzindo drasticamente a necessidade de arquivos de configuração complexos (como XMLs) ao inferir a maior parte do *setup* a partir das bibliotecas (**starters**) incluídas no projeto. É a escolha de fato para construir microsserviços em Java.
- **Exemplo Prático:** Para iniciar um projeto que será uma API REST e se conectará a um banco de dados, o aluno deve acessar o Spring Initializr e selecionar as dependências **Spring Web**, **Spring Data JPA** e o driver do banco de dados de sua preferência (ex: H2 para testes, PostgreSQL ou MySQL para produção).
 - **Vídeo Sugerido 1:** [Building REST services with Spring \(Tutorial\)](#)
 - **Vídeo Sugerido 2:** [How to Build a REST API with Spring Boot: A Step-by-Step Guide - Camunda](#)

Referência do Slide: Criação de Endpoints (@RestController e @PostMapping)

- **Definição:** A anotação **@RestController** é usada para marcar uma classe como um controlador que manipula requisições HTTP, retornando o objeto de domínio (ex:

JSON) diretamente no corpo da resposta. Os métodos dentro dessa classe são mapeados para verbos HTTP específicos, como `@GetMapping` (para leitura) e `@PostMapping` (para criação de novos recursos).

- **Aprofundamento/Complemento (se necessário):** O `@RestController` é uma combinação das anotações `@Controller` e `@ResponseBody`. O uso do `@RequestBody` em um método `@PostMapping` indica que o corpo da requisição (geralmente um JSON) deve ser desserializado para um objeto Java (DTO ou Entidade).
- **Exemplo Prático:**
- Java

```
@RestController
@RequestMapping("/produtos")
public class ProdutoController {
    @PostMapping
    public Produto criarProduto(@RequestBody Produto novoProduto) {
        // Lógica para salvar o novo produto
        return novoProduto;
    }
    @GetMapping
    public List<Produto> listarProdutos() {
        // Lógica para buscar todos os produtos
        return repository.findAll();
    }
}
```

- - **Vídeo Sugerido 1:** [Spring Boot Project: Build a REST API for an E-commerce Platform - YouTube](#)

Referência do Slide: Integração com Banco de Dados (JPA/Spring Data JPA, Repositório)

- **Definição: JPA (Java Persistence API)** é a especificação padrão Java para o mapeamento objeto-relacional (ORM). O **Spring Data JPA** facilita a implementação da camada de acesso a dados (Repositório), reduzindo a necessidade de escrever código clichê (boilerplate) para operações CRUD básicas.
- **Aprofundamento/Complemento (se necessário):** Ao estender a interface `JpaRepository<T, ID>`, o desenvolvedor herda automaticamente métodos como `save()`, `findById()`, `findAll()`, entre outros. Além disso, o Spring Data JPA pode inferir *queries* automaticamente com base nos nomes dos métodos definidos na interface (ex: `findByNomeContaining(String nome)`). A anotação `@Entity` marca a classe de domínio para mapeamento de tabela.
- **Exemplo Prático:**
 - Crie a Entidade: `public class Usuario { @Id @GeneratedValue private Long id; ... }`

- Crie o Repositório: `public interface UsuarioRepository extends JpaRepository<Usuario, Long> {}`.
- Injete o `UsuarioRepository` no `Controller` e use `repository.save(novoUsuario)` para persistir os dados.
- **Vídeo Sugerido 1:** [Spring Boot. Spring Data JPA - Rest CRUD API example - BezKoder](#)

Semana 27 - Aula 2

Tópico Principal da Aula: Desenvolvimento Ágil com Flask (Python)

Subtítulo/Tema Específico: Roteamento, Middleware, Boas Práticas e Testes Unitários

Código da aula: [SIS]ANO2C2B4S27A2

Objetivos da Aula:

- Utilizar **Flask** para desenvolvimento ágilANO2C2B4S27A2.pdf].
- Implementar mecanismos de segurança e lógica transversal usando **Middleware**.
- Organizar rotas de forma modular com **Blueprints**.
- Garantir a qualidade do código com **Testes de Unidade**.

Recursos Adicionais (Sugestão, pode ser adaptado):

- Caderno para anotações;
- Acesso ao laboratório de informática e/ou internet.

Exposição do Conteúdo:

Referência do Slide: Roteamento e Segurança (Middleware)

- **Definição:** O **Flask** é um micro-framework em Python que permite o desenvolvimento web de forma simples e rápida. O **Roteamento** é definido por meio do decorador `@app.route()`, que associa uma URL a uma função (`view function`)ANO2C2B4S27A2.pdf]. **Middleware** (em Flask, implementado via decoradores ou extensões) permite executar lógica transversal, como segurança ou logging, antes ou depois de uma requisição ser processada pela rotaANO2C2B4S27A2.pdf].
- **Aprofundamento/Complemento (se necessário):** O uso de `@app.before_request` no Flask é a forma mais comum de implementar um "middleware" de segurança, permitindo, por exemplo, verificar tokens de autenticação antes que a função da rota seja executada.
- **Exemplo Prático:** Usar o decorador `@app.before_request` para verificar se o cabeçalho `Authorization` está presente na requisição e se o token é válido. Se não for, a execução é abortada e retorna um erro 401 (Não Autorizado).
 - **Vídeo Sugerido 1:** [Curso Flask - Aula 02 - Criando Rotas e configurações - YouTube](#)

Referência do Slide: Boas Práticas de Rotas (Blueprints)

- **Definição:** Os **Blueprints** no Flask são mecanismos para estruturar aplicações maiores, modularizando as rotas em componentes lógicos separados e reutilizáveis [ANO2C2B4S27A2.pdf].
- **Aprofundamento/Complemento (se necessário):** Ao invés de ter um único arquivo gigante (`app.py`) com todas as rotas (`@app.route`), você pode criar um Blueprint para cada domínio da aplicação (ex: `users`, `products`). Isso evita conflitos de código e promove a organização, especialmente em equipes grandes.
- **Exemplo Prático:** Criar um arquivo `usuarios.py` com `usuarios_bp = Blueprint('usuarios', __name__)` e definir todas as rotas de usuário usando `@usuarios_bp.route()`. Depois, registrar esse Blueprint no objeto principal da aplicação: `app.register_blueprint(usuarios_bp, url_prefix='/api/v1/usuarios')`.
 - **Vídeo Sugerido 1:** [Organizando nossa Aplicação com Flask Blueprints (Mencionado na busca 2.7)]

Referência do Slide: Testes de Unidade

- **Definição:** **Testes de Unidade** são testes automatizados que verificam o comportamento de pequenas partes isoladas do código (funções, métodos, rotas) para garantir que funcionem conforme o pretendido. Em Python, o módulo `unittest` é frequentemente utilizado, ou o `pytest`.
- **Aprofundamento/Complemento (se necessário):** No Flask, é boa prática testar se as rotas respondem com o código de status HTTP e o conteúdo corretos. O Flask oferece um **cliente de teste** (`test_client`) que permite simular requisições HTTP sem a necessidade de levantar um servidor web real.
- **Exemplo Prático:** Criar um caso de teste que simula uma requisição GET para a rota `/hello` e verifica se o código de `status` da resposta é `200` (OK) e se o corpo da resposta contém o texto esperado.
 - **Vídeo Sugerido 1:** [Teste e Publicação de API's: Um Guia Definitivo para Iniciantes | Tiago Peniche - DIO](#)
 - **Vídeo Sugerido 2:** [Introdução a testes com Flask - YouTube](#)

Semana 27 - Aula 3

Tópico Principal da Aula: Integração e Comunicação em Microsserviços

Subtítulo/Tema Específico: Arquitetura de Microsserviços, Comunicação Assíncrona, Orquestração e Coreografia

Código da aula: [SIS]ANO2C2B4S27A3

Objetivos da Aula:

- Implementar microsserviços utilizando *frameworks* back-end [ANO2C2B4S27A3.pdf].
- Compreender os desafios da comunicação em arquiteturas distribuídas.

- Diferenciar e aplicar os padrões de **Orquestração** e **Coreografia** para a comunicação.

Recursos Adicionais (Sugestão, pode ser adaptado):

- Caderno para anotações;
- Acesso ao laboratório de informática e/ou internet.

Exposição do Conteúdo:

Referência do Slide: Arquitetura de Microsserviços

- **Definição:** É um estilo arquitetural que estrutura uma aplicação como uma coleção de serviços **pequenos, autônomos e fracamente acoplados**, que se comunicam para cumprir uma funcionalidade de negócio.
- **Aprofundamento/Complemento (se necessário):** Cada microsserviço geralmente é responsável por um único domínio de negócio (*Bounded Context*), possui sua própria base de dados e pode ser desenvolvido e implantado independentemente dos outros serviços. O uso de contêineres (Docker) e orquestradores (Kubernetes) é uma prática padrão para gerenciar a implantação em escala.
- **Exemplo Prático:** Em um sistema bancário, teríamos microsserviços separados para **Conta**, **Transações** e **Empréstimos**, cada um mantido por equipes diferentes e com tecnologia de *stack* que melhor se adapta à sua necessidade.
 - **Vídeo Sugerido 1:** [Estilo da arquitetura de microsserviços - Azure Architecture Center | Microsoft Learn](#)

Referência do Slide: Comunicação entre Microsserviços (Síncrona vs. Assíncrona)

- **Definição:** A comunicação entre microsserviços pode ser **Síncrona** (onde um serviço espera a resposta imediata do outro, geralmente via **REST/HTTP**) ou **Assíncrona** (baseada em mensagens, onde o serviço envia uma mensagem e não espera uma resposta imediata, tipicamente usando *brokers* como Kafka ou RabbitMQ).
- **Aprofundamento/Complemento (se necessário):** A comunicação síncrona é mais simples, mas cria **acoplamento temporal** e pode ser um ponto de falha em cascata. A comunicação assíncrona, embora mais complexa, aumenta a **resiliência** e o **baixo acoplamento**, sendo ideal para arquiteturas orientadas a eventos (*Event-Driven Architectures - EDA*).
- **Exemplo Prático:**
 - **Síncrona:** O serviço de *Interface do Usuário* faz uma chamada **REST (GET)** para o serviço *Produto* para exibir o catálogo.
 - **Assíncrona:** O serviço *Pedido* publica o evento "Pedido Realizado" em uma fila, e o serviço *Estoque* consome essa mensagem para decrementar o item, sem que o serviço *Pedido* precise esperar.
 - **Vídeo Sugerido 1:** [Comunicação entre #Microsserviços usando : Orquestração vs Coreografia - YouTube](#)

Referência do Slide: Orquestração e Coreografia

- **Definição:** São duas estratégias para coordenar fluxos de trabalho que envolvem múltiplos microsserviços.
 - **Orquestração:** Um serviço central (o **Orquestrador**) é responsável por coordenar e dirigir a sequência de chamadas entre os outros serviços (pense em um maestro de orquestra).
 - **Coreografia:** Os serviços agem de forma **descentralizada**, reagindo a eventos uns dos outros, sem uma entidade central de controle (pense em dançarinos de uma coreografia, onde cada um sabe o seu passo em reação ao do colega).
- **Aprofundamento/Complemento (se necessário):** A **Orquestração** oferece **melhor visibilidade** do fluxo, mas cria um alto acoplamento e um potencial ponto único de falha no Orquestrador. A **Coreografia** favorece o **baixo acoplamento** e a resiliência, mas a rastreabilidade e o *troubleshooting* de transações distribuídas se tornam mais desafiadores.
- **Exemplo Prático:**
 - **Orquestração:** Um serviço **Gerenciador de Pedidos** coordena: 1º Chama **Serviço de Estoque**, 2º Chama **Serviço de Pagamento**, 3º Chama **Serviço de Entrega**.
 - **Coreografia:** O serviço **Pedido** envia um evento "PedidoCriado". O **Serviço de Estoque** e o **Serviço de Pagamento** consomem este evento simultaneamente e reagem de forma independente.
 - **Vídeo Sugerido 1:** [Orquestração de contêiner para microsserviços - Azure Architecture Center | Microsoft Learn](#)